Market Gap Analysis

```
import pandas as pd
import numpy as np
import re
from collections import Counter
```

```
!apt-get -qq install -y aria2
!aria2c -x 16 -s 16 -o off.csv.gz "https://static.openfoodfacts.org/data/en.openfoodfacts.org.products.csv.gz"
!ls -lh off.csv.gz
```

```
/sbin/ldconfig.real: /usr/local/lib/libur_loader.so.0 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libhwloc.so.15 is not a symbolic link


03/01 08:20:22 [NOTICE] Downloading 1 item(s)

03/01 08:20:22 [NOTICE] CUID#7 - Redirecting to https://openfoodfacts-ds.s3.eu-west-3.amazonaws.com/en.openfoodfacts.org.produ

03/01 08:20:23 [NOTICE] CUID#9 - Redirecting to https://openfoodfacts-ds.s3.eu-west-3.amazonaws.com/en.openfoodfacts.org.produ

03/01 08:20:23 [NOTICE] CUID#13 - Redirecting to https://openfoodfacts-ds.s3.eu-west-3.amazonaws.com/en.openfoodfacts.org.produ

03/01 08:20:23 [NOTICE] CUID#12 - Redirecting to https://openfoodfacts-ds.s3.eu-west-3.amazonaws.com/en.openfoodfacts.org.produ

03/01 08:20:23 [NOTICE] CUID#21 - Redirecting to https://openfoodfacts-ds.s3.eu-west-3.amazonaws.com/en.openfoodfacts.org.produ

03/01 08:20:23 [NOTICE] CUID#19 - Redirecting to https://openfoodfacts-ds.s3.eu-west-3.amazonaws.com/en.openfoodfacts.org.produ

03/01 08:20:23 [NOTICE] CUID#14 - Redirecting to https://openfoodfacts-ds.s3.eu-west-3.amazonaws.com/en.openfoodfacts.org.produ

03/01 08:20:23 [NOTICE] CUID#15 - Redirecting to https://openfoodfacts-ds.s3.eu-west-3.amazonaws.com/en.openfoodfacts.org.produ

03/01 08:20:23 [NOTICE] CUID#23 - Redirecting to https://openfoodfacts-ds.s3.eu-west-3.amazonaws.com/en.openfoodfacts.org.produ

03/01 08:20:23 [NOTICE] CUID#10 - Redirecting to https://openfoodfacts-ds.s3.eu-west-3.amazonaws.com/en.openfoodfacts.org.produ

03/01 08:20:23 [NOTICE] CUID#11 - Redirecting to https://openfoodfacts-ds.s3.eu-west-3.amazonaws.com/en.openfoodfacts.org.produ

03/01 08:20:24 [ERROR] CUID#16 - Download aborted. URI=https://static.openfoodfacts.org/data/en.openfoodfacts.org.products.csv
Exception: [SocketCore.cc:885] errorCode=1 Failed to receive data, cause: The TLS connection was non-properly terminated.

03/01 08:20:24 [ERROR] CUID#18 - Download aborted. URI=https://static.openfoodfacts.org/data/en.openfoodfacts.org.products.csv
Exception: [SocketCore.cc:885] errorCode=1 Failed to receive data, cause: The TLS connection was non-properly terminated.

03/01 08:20:24 [ERROR] CUID#22 - Download aborted. URI=https://static.openfoodfacts.org/data/en.openfoodfacts.org.products.csv
Exception: [SocketCore.cc:885] errorCode=1 Failed to receive data, cause: The TLS connection was non-properly terminated.

03/01 08:20:24 [ERROR] CUID#20 - Download aborted. URI=https://static.openfoodfacts.org/data/en.openfoodfacts.org.products.csv
Exception: [SocketCore.cc:885] errorCode=1 Failed to receive data, cause: The TLS connection was non-properly terminated.

03/01 08:20:24 [ERROR] CUID#23 - Download aborted. URI=https://static.openfoodfacts.org/data/en.openfoodfacts.org.products.csv
Exception: [SocketCore.cc:885] errorCode=1 Failed to receive data, cause: The TLS connection was non-properly terminated.

03/01 08:20:25 [ERROR] CUID#17 - Download aborted. URI=https://static.openfoodfacts.org/data/en.openfoodfacts.org.products.csv
Exception: [SocketCore.cc:885] errorCode=1 Failed to receive data, cause: The TLS connection was non-properly terminated.

03/01 08:20:31 [NOTICE] Download complete: /content/off.csv.gz

Download Results:
gid   |stat|avg speed  |path/URI
======+====+===========+=======================================================
7359ce|OK  |   146MiB/s|/content/off.csv.gz

Status Legend:
(OK):download completed.
-rw-r--r-- 1 root root 1.1G Mar  1 08:20 off.csv.gz
```

```
COLUMNS = ["product_name","categories_tags","sugars_100g","proteins_100g","ingredients_text"]

TARGET_ROWS = 60_000
CHUNKSIZE = 10_000

chunks = []
total = 0
```

```
reader = pd.read_csv(
    "off.csv.gz",
    sep="\t",
    usecols=COLUMNS,
    chunksize=CHUNKSIZE,
    low_memory=True,
    on_bad_lines="skip"
)

for c in reader:
    chunks.append(c)
    total += len(c)
    if total >= TARGET_ROWS:
        break

df_raw = pd.concat(chunks, ignore_index=True)
df_raw.shape
```

```
(60000, 5)
```

```
df = df_raw.copy()

df["sugars_100g"] = pd.to_numeric(df["sugars_100g"], errors="coerce")
df["proteins_100g"] = pd.to_numeric(df["proteins_100g"], errors="coerce")

# Drop missing critical fields
df = df.dropna(subset=["product_name","sugars_100g","proteins_100g"])

# Remove biologically impossible values
df = df[
    df["sugars_100g"].between(0, 100) &
    df["proteins_100g"].between(0, 100)
].copy()

df.shape
```

```
(1174, 5)
```

```
df["first_category"] = df["categories_tags"].astype("string").str.split(",").str[0]
df["first_category"].value_counts().head(20)
```

first_category	count
en:plant-based-foods-and-beverages	433
en:dairies	404
en:undefined	22
en:snacks	20
en:condiments	8
en:beverages-and-beverages-preparations	8
en:null	8
en:meats-and-their-products	7
en:seafood	6
en:beverages	3
en:meals	2
en:canned-foods	2
en:farming-products	2
en:salted-snacks	2
en:frozen-foods	1
en:dietary-supplements	1
en:dairy-free-sour-cream	1
en:petit-dejeuners	1
en:chips-and-fries	1
en:cottage-pie	1

dtype: int64

```
def map_first_category(cat):
    if pd.isna(cat):
        return "Other"
    cat = str(cat).lower()

    if "snacks" in cat:
        return "Snacks"
    if "desserts" in cat:
        return "Desserts"
    if "beverages" in cat:
        return "Beverages"
    if "dairies" in cat:
        return "Dairy"
    if "condiments" in cat or "sauces" in cat:
        return "Condiments & Sauces"
    if "frozen" in cat:
        return "Frozen Foods"
    if "breakfast" in cat or "cereals" in cat:
        return "Cereals & Breakfast"
    if "plant-based" in cat:
        return "Plant-Based Foods"
    if "meals" in cat:
        return "Prepared Meals"
    if "meats" in cat or "seafood" in cat:
        return "Meat & Seafood"
    if cat in ["en:undefined", "en:null"]:
        return "Other"

    return "Other"

df["primary_category"] = df["first_category"].apply(map_first_category)
df["primary_category"].value_counts()
```

primary_category	count
Beverages	444
Dairy	404
Other	279
Snacks	22
Meat & Seafood	13
Condiments & Sauces	8
Prepared Meals	2
Frozen Foods	1
Desserts	1

dtype: int64

```
HP_MIN = 10
LS_MAX = 5

df["is_opportunity"] = (df["proteins_100g"] > HP_MIN) & (df["sugars_100g"] < LS_MAX)
blue_ocean = df[df["is_opportunity"]].copy()

blue_ocean["primary_category"].value_counts()
```

primary_category	count
Other	71
Beverages	69
Dairy	13
Meat & Seafood	12
Frozen Foods	1
Snacks	1
Condiments & Sauces	1

dtype: int64

```
total_counts = df["primary_category"].value_counts()
blue_counts = blue_ocean["primary_category"].value_counts()

analysis = pd.DataFrame({
    "Total Products": total_counts,
    "HighProtein_LowSugar": blue_counts
}).fillna(0)

analysis["Opportunity_Ratio"] = analysis["HighProtein_LowSugar"] / analysis["Total Products"]
analysis.sort_values("Opportunity_Ratio", ascending=True)
```

primary_category	Total Products	HighProtein_LowSugar	Opportunity_Ratio
Desserts	1	0.0	0.000000
Prepared Meals	2	0.0	0.000000
Dairy	404	13.0	0.032178
Snacks	22	1.0	0.045455
Condiments & Sauces	8	1.0	0.125000
Beverages	444	69.0	0.155405
Other	279	71.0	0.254480
Meat & Seafood	13	12.0	0.923077
Frozen Foods	1	1.0	1.000000

Gemini

```
snack_blue = blue_ocean[blue_ocean["primary_category"
snack_blue[["proteins_10                0g","sugars_10
snack_blue[["proteins_100g","sugars_100g"]].describe(
```

	proteins_100g	sugars_100g
count	1.0	1.000
mean	12.5	3.125
std	NaN	NaN
min	12.5	3.125
25%	12.5	3.125
50%	12.5	3.125
75%	12.5	3.125
max	12.5	3.125

```
kpi_table = (
    df.groupby("primary_category")
      .agg(
          total_products=("product_name", "count"),
          opportunity_products=("is_opportunity", "sum"),
          opportunity_density=("is_opportunity", "mean"),
      )
      .reset_index()
)

kpi_table["opportunity_density"] = (kpi_table["opportunity_density"] * 100).round(2)
kpi_table.sort_values("opportunity_density", ascending=False).head(10)
```

	primary_category	total_products	opportunity_products	opportunity_density
4	Frozen Foods	1	1	100.00
5	Meat & Seafood	13	12	92.31
6	Other	279	71	25.45
0	Beverages	444	69	15.54
1	Condiments & Sauces	8	1	12.50
8	Snacks	22	1	4.55
2	Dairy	404	13	3.22
3	Desserts	1	0	0.00
7	Prepared Meals	2	0	0.00

```
kpi_table.to_csv("kpi_opportunity_density.csv", index=False)
```

```
!ls -lh kpi_opportunity_density.csv
```

```
-rw-r--r-- 1 root root 269 Mar  1 08:44 kpi_opportunity_density.csv
```

```
from google.colab import files
files.download("kpi_opportunity_density.csv")
```

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.